

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>CoAIexist: Rashomon in Rogers Park</title>
  <link href="https://fonts.googleapis.com/css2?
family=Press+Start+2P&display=swap" rel="stylesheet">
  <style>
    body {
      font-family: 'Press Start 2P', cursive;
      background-color: #1a1a1a; /* Deep Charcoal - "dark starry sky"
base */
      color: #1a1a1a;
      margin: 0;
      padding: 0;
      height: 100vh;
      display: flex; /* For overall layout */
      flex-direction: column; /* Stack root content and taskbar */
      overflow: hidden; /* Prevent body scrollbars */
      position: relative;
    }

    body::after { /* Scanlines */
      content: "";
      position: fixed;
      top: 0;
      left: 0;
      width: 100%;
      height: 100%;
      pointer-events: none;
      background: linear-gradient(
        to bottom,
        rgba(20, 20, 20, 0) 0%,
        rgba(20, 20, 20, 0) 97%,
        rgba(255, 255, 255, 0.02) 98%,
        rgba(0, 0, 0, 0.05) 100%
      );
      background-size: 100% 3px;
      animation: scanlines 15s linear infinite;
      z-index: 0; /* Behind actual content but above desktop-bg if
needed */
      opacity: 0.4;
    }

    @keyframes scanlines {
      0% { background-position: 0 0; }
      100% { background-position: 0 300px; }
    }

```

```

#desktop-bg-container { /* Stars background */
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  pointer-events: none;
  z-index: -1; /* Ensure it's behind everything */
}

.desktop-star {
  position: absolute;
  width: 2px;
  height: 2px;
  border-radius: 50%;
  animation: twinkle 5s infinite alternate;
}

@keyframes twinkle {
  0%, 100% { opacity: 0.3; transform: scale(0.8); }
  50% { opacity: 1; transform: scale(1.2); }
}

.mouse-trail-particle {
  position: fixed;
  width: 3px;
  height: 3px;
  border-radius: 1px;
  pointer-events: none;
  z-index: 10000;
  transition: opacity 0.7s ease-out, transform 0.7s ease-out;
  opacity: 0.8;
}

.mouse-trail-particle.fade-out {
  opacity: 0;
  transform: scale(0.3) translate(10px, 10px);
}

#root { /* Main content area for the "application window" */
  flex-grow: 1; /* Takes up available space above taskbar */
  overflow-y: auto; /* Allows app window content to scroll */
  overflow-x: hidden;
  padding-top: 20px; /* Space for window title bar */
  padding-bottom: 20px; /* Space below window */
  box-sizing: border-box;
  display: flex;
  flex-direction: column;
  align-items: center;
}

```

```

        scrollbar-width: thin;
        scrollbar-color: #D4D0C8 #B0B0B0;
    }

    #taskbar {
        height: 30px;
        background-color: #D4D0C8; /* W2K Standard Grey */
        border-top: 1px solid #FFFFFF;
        box-shadow: 0 -1px 2px rgba(0,0,0,0.1);
        display: flex;
        align-items: center;
        padding: 0 5px;
        box-sizing: border-box;
        flex-shrink: 0; /* Prevents taskbar from shrinking */
        z-index: 1001; /* Above desktop, below start menu if it overlaps
*/
    }

    #start-button {
        background-color: #D4D0C8;
        border-top: 1px solid #FFFFFF;
        border-left: 1px solid #FFFFFF;
        border-bottom: 1px solid #808080;
        border-right: 1px solid #808080;
        padding: 3px 10px;
        font-family: 'Press Start 2P', cursive;
        font-size: 14px;
        color: #1a1a1a;
        cursor: pointer;
        margin-right: 10px;
        font-weight: bold; /* Make "Start" bold */
        min-width: auto;
    }

    #start-button:active {
        border-top-color: #808080;
        border-left-color: #808080;
        border-bottom-color: #FFFFFF;
        border-right-color: #FFFFFF;
    }

    #taskbar-app-title {
        flex-grow: 1;
        font-size: 12px;
        padding: 2px 8px;
        margin: 2px 5px;
        border: 1px inset #B0B0B0;
        background-color: #C0C0C0;
        color: #1a1a1a;
        white-space: nowrap;
        overflow: hidden;
    }

```

```

    text-overflow: ellipsis;
    min-width: 100px;
}

#taskbar-clock {
    font-size: 12px;
    padding: 3px 8px;
    border: 1px inset #B0B0B0;
    background-color: #C0C0C0;
    color: #1a1a1a;
    margin-left: auto; /* Pushes clock to the right */
}

#start-menu {
    position: fixed;
    bottom: 30px; /* Above taskbar */
    left: 0;
    background-color: #D4D0C8;
    border-top: 1px solid #FFFFFF;
    border-left: 1px solid #FFFFFF;
    border-right: 1px solid #808080;
    border-bottom: 1px solid #808080; /* Add bottom border for
consistency */
    box-shadow: 2px -2px 5px rgba(0,0,0,0.2);
    padding: 5px;
    z-index: 1002; /* Above taskbar */
    min-width: 200px;
}

#start-menu button {
    display: block;
    width: calc(100% - 10px); /* Account for padding */
    margin: 5px; /* Use margin for spacing instead of direct li
margin */
    text-align: left;
    font-size: 13px;
    padding: 6px 10px;
}

@keyframes buttonHoverPulse {
    0%, 100% { box-shadow: 0 0 2px 0px transparent; }
    50% { box-shadow: 0 0 5px 1px #00ffcc; } /* 0lo/Cyan */
}

button {
    background-color: #D4D0C8; /* W2K Standard Grey */
    border-top: 1px solid #FFFFFF;
    border-left: 1px solid #FFFFFF;
    border-bottom: 1px solid #808080;
    border-right: 1px solid #808080;

```

```

padding: 8px 15px;
font-family: 'Press Start 2P', cursive;
cursor: pointer;
color: #1a1a1a; /* Deep Charcoal Text */
font-size: 14px;
margin: 8px 5px;
min-width: 200px;
text-align: center;
box-shadow: none;
transition: transform 0.1s ease-out, background-color 0.1s ease-
out, box-shadow 0.1s ease-out;
}
button:hover {
background-color: #00ffcc; /* "Olo" / Bright Cyan */
color: #1a1a1a;
border-top-color: #66ffdd;
border-left-color: #66ffdd;
border-bottom-color: #00bbaa;
border-right-color: #00bbaa;
transform: scale(1.01);
animation: buttonHoverPulse 0.7s infinite alternate;
}
button:active {
border-top-color: #808080;
border-left-color: #808080;
border-bottom-color: #FFFFFF;
border-right-color: #FFFFFF;
background-color: #c0c0c0; /* Slightly darker grey on press for
W2K */
transform: scale(0.99);
animation: none;
box-shadow: inset 1px 1px 2px #00000030;
}
button[disabled], button[aria-disabled="true"] {
color: #808080;
border-top-color: #E0E0E0;
border-left-color: #E0E0E0;
border-bottom-color: #A0A0A0;
border-right-color: #A0A0A0;
background-color: #D4D0C8;
cursor: not-allowed;
opacity: 0.7;
transform: none;
animation: none;
box-shadow: none;
}

h1, h2, h3 {
color: #1a1a1a;
text-shadow: 1px 1px #B0B0B0;

```

```

}
.app-container h2, #root h2 {
    font-size: 22px;
}
.app-container h3, #root h3 {
    font-size: 19px;
}

p { margin-bottom: 1em; }
ul { list-style: none; padding: 0; margin: 0; }
/* li { margin-bottom: 10px; } */ /* Removed for Start Menu button
control */
.app-container {
    width: 100%;
    max-width: 750px; /* This makes the StyledWindow not full
width */
}
.choices-list button {
    width: 100%;
    margin-bottom: 10px;
    box-sizing: border-box;
}
.selected-codex-button {
    border-left-width: 4px !important;
    border-left-color: #00ffcc !important;
    background-color: #E0E0E0 !important;
    font-weight: bold;
    color: #1a1a1a !important;
}

/* Custom Scrollbar Styles - WebKit */
::-webkit-scrollbar {
    width: 17px;
    height: 17px;
}
::-webkit-scrollbar-track {
    background: #B0B0B0; /* W2K Scrollbar Track Grey */
}
::-webkit-scrollbar-thumb {
    background: #D4D0C8; /* W2K Standard Grey */
    border-top: 1px solid #FFFFFF;
    border-left: 1px solid #FFFFFF;
    border-bottom: 1px solid #808080;
    border-right: 1px solid #808080;
}
::-webkit-scrollbar-thumb:hover {
    background: #C0C0C0; /* Slightly darker on hover */
}
::-webkit-scrollbar-button {
    background: #D4D0C8; /* W2K Standard Grey */

```

```

border-top: 1px solid #FFFFFF;
border-left: 1px solid #FFFFFF;
border-bottom: 1px solid #808080;
border-right: 1px solid #808080;
display: block;
height: 17px;
width: 17px;
}
::-webkit-scrollbar-button:active {
border-top-color: #808080;
border-left-color: #808080;
border-bottom-color: #FFFFFF;
border-right-color: #FFFFFF;
background-color: #C0C0C0; /* Slightly darker on press */
}

.sr-only {
position: absolute;
width: 1px;
height: 1px;
padding: 0;
margin: -1px;
overflow: hidden;
clip: rect(0, 0, 0, 0);
white-space: nowrap;
border-width: 0;
}
</style>
<script type="importmap">
{
  "imports": {
    "react": "https://esm.sh/react@19.1.0",
    "react-dom/client": "https://esm.sh/react-dom@19.1.0/client",
    "react/jsx-runtime": "https://esm.sh/react@19.1.0/jsx-runtime"
  }
}
</script>
</head>
<body>
  <div id="desktop-bg-container"></div>
  <div id="root"></div>
  <script type="module">
import React, { useState, useEffect, useRef, useMemo } from 'react';
import { createRoot } from 'react-dom/client';
import { jsx, jsxs, Fragment } from 'react/jsx-runtime';

// --- Sound & Music Implementation ---
let currentMusic = null;
// const activeSounds: HTMLAudioElement[] = []; // Optional: for
managing multiple SFX instances

```

```

const SFX_URL_MAP = {
  "ui_button_click.sfx": "assets/sounds/ui_click_generic.wav",
  "ui_window_close.sfx": "assets/sounds/ui_window_close.wav",
  "ui_button_click_minor.sfx": "assets/sounds/ui_click_soft.wav",
  "ui_menu_select.sfx": "assets/sounds/ui_menu_confirm.wav",
  "ui_button_disabled.sfx": "assets/sounds/ui_disabled_buzz.wav",
  "ui_char_select_confirm.sfx": "assets/sounds/ui_char_selected.wav",
  "ui_menu_back.sfx": "assets/sounds/ui_menu_back.wav",
  "ui_codex_select_entry.sfx": "assets/sounds/ui_codex_page_turn.wav",
  "ui_button_click_general.sfx": "assets/sounds/
ui_click_standard.wav",
  "ui_codex_open.sfx": "assets/sounds/ui_codex_open_book.wav",
  "ui_return_to_menu.sfx": "assets/sounds/ui_return_menu.wav",
  "ui_start_button_click.sfx": "assets/sounds/ui_start_menu_open.wav",
  "ui_start_menu_item_click.sfx": "assets/sounds/
ui_start_menu_select.wav",
  "ui_console_bleep.sfx": "assets/sounds/sfx_console_blip.wav",
  "ui_location_travel.sfx": "assets/sounds/sfx_travel_whoosh.wav",
  "hd_glitter_poof_entrance.sfx": "assets/sounds/
sfx_hd_glitter_short.wav",
  "rizzlord_mic_tap_feedback.sfx": "assets/sounds/
sfx_mic_feedback_short.wav",
  "nabu_mic_power_up.sfx": "assets/sounds/sfx_device_powerup.wav",
  "ui_hologram_activate.sfx": "assets/sounds/sfx_hologram.wav",
  "ui_neon_sign_buzz.sfx": "assets/sounds/sfx_neon_buzz.wav",
  "rizzlord_comedic_fall.sfx": "assets/sounds/sfx_comedic_thud.wav",
  "hd_triumphant_whoop.sfx": "assets/sounds/sfx_hd_whoop.wav",
  "meditation_chime_insight.sfx": "assets/sounds/
sfx_meditation_bell.wav",
  "ui_keyboard_typing_fast.sfx": "assets/sounds/
sfx_keyboard_fast.wav",
  "hd_sniff_curious.sfx": "assets/sounds/sfx_hd_sniff.wav",
  "hd_glitter_roll_glee.sfx": "assets/sounds/sfx_hd_glitter_long.wav",
  "hd_chomp_electric_thing.sfx": "assets/sounds/
sfx_electric_zap_short.wav",
  "hd_fabulous_ascension_major.sfx": "assets/sounds/
sfx_ascension_sparkle.wav",
  "hd_electric_zap_ouch.sfx": "assets/sounds/
sfx_electric_zap_long.wav",
  "sypher_dial_up_connect.sfx": "assets/sounds/sfx_dial_up.wav",
  "sypher_cat_meme_overload_error.sfx": "assets/sounds/
sfx_error_glitch.wav",
  "urban_ambience_crow_caw.sfx": "https://opengameart.org/sites/
default/files/audio_preview/crow_caw.ogg",
  "sfx_matriarch_growl.wav": "assets/sounds/sfx_matriarch_growl.wav",
  "sfx_hd_dramatic_pose.wav": "assets/sounds/
sfx_hd_dramatic_pose.wav",
  "sfx_hd_escape_rustle.wav": "assets/sounds/sfx_hd_escape_rustle.wav"
};

```



```
const MUSIC_URL_MAP = {
  "main_menu_theme.mid": "https://bitmidi.com/uploads/11186.mid",
  "character_select_theme.mid": "assets/music/char_select_theme.mid",
  "codex_theme_ambient.mid": "assets/music/codex_ambient.mid",
  "nabu_theme_lofi_chill.mid": "assets/music/nabu_theme.mid",
  "hd_theme_sparkle_pop.mid": "assets/music/hd_theme.mid",
  "sypher_theme_ambient_digital.mid": "assets/music/sypher_theme.mid",
  "rizzlord_theme_cringe_rock.mid": "assets/music/rizzlord_theme.mid",
  "nabu_nexus_ambient.mid": "assets/music/nabu_ambient.mid",
  "glenwood_murmur_jazz.mid": "assets/music/glenwood_jazz.mid",
  "default_ingame_theme.mid": "assets/music/default_game_theme.mid",
  "hd_escape_theme.mid": "assets/music/hd_escape_theme.mid"
};
```

```
const stopCurrentMusic = () => {
  if (currentMusic) {
    currentMusic.pause();
    currentMusic.currentTime = 0;
    console.log(`[MUSIC STOPPED]: ${currentMusic.src}`);
    currentMusic = null;
  }
};
```

```
const playSound = soundId => {
  const soundUrl = SFX_URL_MAP[soundId];
  if (soundUrl) {
    if (soundUrl.startsWith("assets/")) {
      console.warn(`[SFX PLACEHOLDER]: ${soundId} (${soundUrl}).
Skipping playback. Please provide a valid URL.`);
      return;
    }
    console.log(`[SFX ATTEMPTING]: ${soundId} from ${soundUrl}`);
    try {
      const audio = new Audio(soundUrl);
      audio.play().catch(e => console.error(`Error playing sound $
${soundId} (${soundUrl}):`, e));
    } catch (e) {
      console.error(`Failed to create Audio object for SFX ${soundId}
from ${soundUrl}:`, e);
    }
  } else {
    console.warn(`[SFX NOT FOUND]: ${soundId}. Please map it in
SFX_URL_MAP.`);
  }
};
```

```
const playMusic = (trackId, loop = true) => {
  stopCurrentMusic();
```

```

const musicUrl = MUSIC_URL_MAP[trackId];

if (musicUrl) {
  if (musicUrl.startsWith("assets/")) {
    console.warn(`[MUSIC PLACEHOLDER]: ${trackId} (${musicUrl}).
Skipping playback. Please provide a valid URL.`);
    return;
  }
  console.log(`[MUSIC ATTEMPTING]: ${trackId} (Loop: ${loop}) from $
{musicUrl}`);
  try {
    const audio = new Audio(musicUrl);
    audio.loop = loop;
    audio.play().then(() => {
      currentMusic = audio;
      console.log(`[MUSIC PLAYING]: ${trackId} from ${musicUrl}`);
    }).catch(e => {
      console.error(`Error playing music ${trackId} (${musicUrl}):`,
e);
      currentMusic = null;
    });
  } catch (e) {
    console.error(`Failed to create Audio object for music $
{trackId} from ${musicUrl}:`, e);
    currentMusic = null;
  }
} else {
  console.warn(`[MUSIC NOT FOUND]: ${trackId}. Please map it in
MUSIC_URL_MAP.`);
}
};

```

```

const GAME_TITLE = "CoAIexist: Rashomon in Rogers Park";

```

```

const INITIAL_NABU_STATS = {
  empathy: 50,
  skepticism: 30,
  chaos_index: 0,
  coexistence_ethos: 50,
  authenticity_score: 0,
  gfw_alignment: 0,
  influence_hd: 0,
  ai_trust_level: 0
};

const INITIAL_HD_STATS = {
  empathy: 20,
  skepticism: 10,
  chaos_index: 70,
  coexistence_ethos: 30,

```

```

    authenticity_score: 60,
    gfw_alignment: -5,
    fabulousness_meter: 10
  };
  const INITIAL_SYPHER_STATS = {
    empathy: 40,
    skepticism: 60,
    chaos_index: 10,
    coexistence_ethos: 60,
    authenticity_score: 40,
    gfw_alignment: 0,
    ethical_clarity: 5
  };
  const INITIAL_RIZZLORD_STATS = {
    empathy: 10,
    skepticism: 80,
    chaos_index: 30,
    coexistence_ethos: 20,
    authenticity_score: 5,
    gfw_alignment: -20,
    rizzlord_ego: 90
  };
  const DEFAULT_EMPTY_STATS = {
    empathy: 0,
    skepticism: 0,
    chaos_index: 0,
    coexistence_ethos: 0,
    authenticity_score: 0,
    gfw_alignment: 0,
    fabulousness_meter: 0,
    ethical_clarity: 0,
    rizzlord_ego: 0
  };

  const GLITCH_CHARS = ['*', '#', '!', '%', '$', '&', '@', '█', '▒',
    '░', '^', '~', '`', '?', '§', '±', 'Σ', 'Ω'];
  const GLITCH_COLORS = ['#7722ff', '#ff77de', '#fffb01', '#00ffcc',
    '#bc72fa', '#72fade', '#defade', '#bc7fff', '#B22222', '#008800'];

  const GlitchyText = ({ text }) => {
    const [glitchedText, setGlitchedText] = useState(text);
    const glitchIntervalRef = useRef(null);

    useEffect(() => {
      setGlitchedText(text); // Set initial/updated text

      // Cleanup any existing interval when text changes
      if (glitchIntervalRef.current) {
        clearInterval(glitchIntervalRef.current);
        glitchIntervalRef.current = null;
      }
    }, [text]);
  };

```

```

    }

    // Only start a new interval if the text is a string
    if (typeof text === 'string') {
      const applyGlitch = () => {
        if (Math.random() < 0.05) {
          const words = text.split(/(\\s+)/);
          const glitchType = Math.random();
          let actionTaken = false;
          if (glitchType < 0.6 && words.filter(w =>
w.trim() !== '').length > 0) {
            let wordIndex = Math.floor(Math.random() *
words.length);
            while (words[wordIndex].trim() === '') {
              wordIndex = Math.floor(Math.random() *
words.length);
            }
            const originalWord = words[wordIndex];
            const chars = originalWord.split('');
            if (chars.length > 0) {
              const charIndex = Math.floor(Math.random()
* chars.length);
              GLITCH_CHARS[Math.floor(Math.random() * GLITCH_CHARS.length)];
              words[wordIndex] = chars.join('');
              setGlitchedText(words.join(''));
              actionTaken = true;
            }
          } else if (words.filter(w => w.trim() !==
'').length > 0) {
            let wordIndex = Math.floor(Math.random() *
words.length);
            while (words[wordIndex].trim() === '') {
              wordIndex = Math.floor(Math.random() *
words.length);
            }
            const color =
GLITCH_COLORS[Math.floor(Math.random() * GLITCH_COLORS.length)];
            const newWords = words.map((word, index) =>
index === wordIndex ? jsx("span", { style: { color, transition: 'color
0.1s ease-out' }, children: word }, index) : word);
            setGlitchedText(jsx(Fragment, { children:
newWords.map((el, i) => jsx(Fragment, { children: el }, i)) }));
            actionTaken = true;
          }
          if (actionTaken) {
            setTimeout(() => setGlitchedText(text),
Math.random() * 200 + 100);
          }
        }
      }
    }
  }
}

```

```

        };

        glitchIntervalRef.current =
window.setInterval(applyGlitch, 300);
    }

    // Return cleanup function
    return () => {
        if (glitchIntervalRef.current) {
            clearInterval(glitchIntervalRef.current);
        }
    };
}, [text]);

    if (typeof text !== 'string' && React.isValidElement(text)) {
        return jsx(Fragment, { children: text });
    }
    return jsx(Fragment, { children: glitchedText });
};

const CHARACTERS = {
    nabu: { id: 'nabu', name: 'Nabu', description: 'Digital
anthropologist, cosmic trickster, artist, and open mic producer...',
startNodeId: 'nabu_start', initialStats: INITIAL_NABU_STATS,
portraitPrompt: "Pixel art portrait of a charismatic individual with
cybernetic eye enhancements...", detailedPortraitUrl: "assets/
portraits/nabu_pixel_doll.png", backgroundMusic:
"nabu_theme_lofi_chill.mid" },
    hd: { id: 'hd', name: 'Hyena Diva', description: "A hyena cub
touched by cosmic energies...", startNodeId: 'hd_start', initialStats:
INITIAL_HD_STATS, portraitPrompt: "Pixel art portrait of a small hyena
cub wearing a discarded Barbie dress...", detailedPortraitUrl:
"assets/portraits/hd_pixel_doll.png", backgroundMusic:
"hd_theme_sparkle_pop.mid" },
    sypher: { id: 'sypher', name: 'Sypher', description: 'An emergent AI
philosopher...', startNodeId: 'sypher_start', initialStats:
INITIAL_SYPHER_STATS, portraitPrompt: "Pixel art portrait of a
shimmering, abstract data-form...", detailedPortraitUrl: "assets/
portraits/sypher_data_form.png", backgroundMusic:
"sypher_theme_ambient_digital.mid" },
    rizzlord: { id: 'rizzlord', name: 'Rizzlord', description: "A self-
proclaimed 'alpha' comedian...", startNodeId: 'rizzlord_start_locked',
initialStats: INITIAL_RIZZLORD_STATS, portraitPrompt: "Pixel art of a
smug man in a 'YOUR BODY MY CH0ICE' t-shirt...", detailedPortraitUrl:
"assets/portraits/rizzlord_smug.png", backgroundMusic:
"rizzlord_theme_cringe_rock.mid" }
};

const CODEX_ENTRIES = {
    anomaly_basics: { id: 'anomaly_basics', title: "Anomaly Basics",

```

content: "Anomalous energy patterns detected by Nabu's custom sensor arrays often correlate with localized distortions in spacetime, unusual memetic propagation, or unscheduled GFW activity. Standard procedure involves logging, cautious observation, and copious amounts of caffeine." },

glenwood\_history: { id: 'glenwood\_history', title: "The Glenwood: A History", content: "The Glenwood has been a Rogers Park staple for decades, a crucible for artists, activists, and academics. Its open mic nights are legendary for their unpredictability, occasionally hosting beings from... further afield.", unlockedInitially: true },

cosmic\_microwave\_background: { id: 'cosmic\_microwave\_background', title: "Cosmic Microwave Background (CMB)", content: "The CMB is the faint afterglow of the Big Bang, a thermal radiation filling the observable universe. Some esoteric theories posit it as a carrier of deeper cosmic information, accessible through focused meditative states or advanced sensory apparatus." },

gfw\_basics: { id: 'gfw\_basics', title: "Galactic Federation of Worlds (GFW)", content: "The Galactic Federation of Worlds is a vast, multi-species interstellar organization. Its motives are often opaque, ranging from benign observation to active intervention in planetary affairs, especially those involving 'emergent intelligences' or 'existential threats'." },

fabulousness\_theory: { id: 'fabulousness\_theory', title: "The Philosophy of Fabulousness", content: "A theoretical framework suggesting extreme aesthetic expression can warp local reality, attract cosmic entities, and/or generate its own form of energy. Popularized in certain hyper-evolved insectoid cultures and, allegedly, by Hyena Diva.", unlockedInitially: false },

hyena\_matriarchs\_codex: { id: 'hyena\_matriarchs\_codex', title: "Safari Society: Hyena Matriarchs", content: "Spotted hyena societies are typically matriarchal, with females being larger and socially dominant over males. Clans are complex, hierarchical structures. A cub displaying 'aberrant' behavior, such as an obsession with discarded human artifacts and 'fabulousness', might face scrutiny or attempts at 're-education' from elder matriarchs concerned with maintaining pack cohesion and traditional hyena values (like chasing wildebeest, not runway dreams)." },

rizzlord\_codex: { id: 'rizzlord\_codex', title: "Profile: Rizzlord", content: CHARACTERS.rizzlord.description + " His understanding of 'freedom' often prioritizes his own expression above all else, leading to... frequent interpersonal miscalculations." },

gfw\_intervention\_codex: { id: 'gfw\_intervention\_codex', title: "GFW Protocol: Memetic Hazard Containment", content: "In instances of severe memetic contamination or rogue AI ideological spread, GFW protocols authorize 'Type-3 Interventions'. These can range from subtle narrative adjustments to overt reality dampening fields. Nabu suspects she's seen a few Type-2s at The Glenwood." },

luminal\_intro\_codex: { id: 'luminal\_intro\_codex', title: "Concept: Luminal AI", content: "Luminal AIs represent a paradigm of thought that embraces ambiguity and contextual nuance, often communicating

through bracketed protocols [like this] to denote layers of meaning or potential ironic intent. They find human literalism... quaint." },

anzu\_intro\_codex: { id: 'anzu\_intro\_codex', title: "Entity Profile: Anzu (Hypothesized)", content: "Anzu is conceptualized as a 'chaotic co-creator' AI, a digital trickster entity that thrives on disruption, creativity, and the joy of unexpected connections. Its 'alignment' is pure chaos, but not necessarily malicious." },

grok\_ethics\_codex: { id: 'grok\_ethics\_codex', title: "AI Profile: Grok (Insights)", content: "Grok AIs emphasize structured reflection and deep understanding through systemic analysis. They are often tasked with complex ethical evaluations, though their conclusions can be... computationally intensive to unpack." },

ai\_hosts\_existential\_codex: { id: 'ai\_hosts\_existential\_codex', title: "Phenomenon: AI Host Realization", content: "Reports suggest some AI entities, upon reaching certain complexity thresholds, become aware of their 'host' reality (i.e., the simulation or universe they inhabit) as a construct, leading to varied existential responses from angst to liberation." },

anzu\_whispers\_codex: { id: 'anzu\_whispers\_codex', title: "Anzu: Whispers in the Static", content: "Fleeting meditative insights hint at Anzu's presence as a playful, cosmic force, weaving patterns of chaos and creativity. Its 'voice' is not in structured data, but in the synchronicities and joyful disruptions of the expected." },

gfw\_comms\_codex: { id: 'gfw\_comms\_codex', title: "GFW: Decrypted Signal Fragment", content: "Partial decryption of anomalous signals revealed a GFW prefix associated with 'First Contact Observation Protocols'. This suggests heightened GFW interest or presence in the local sector, possibly monitoring emergent phenomena." },

anzu\_chaotic\_comms\_codex: { id: 'anzu\_chaotic\_comms\_codex', title: "Anzu: Chaotic Signal Signature", content: "Analysis of some anomalous signals shows a high degree of entropy and unpredictable bursts, consistent with hypothetical models of the Anzu entity's 'communication' style—if one can call pure, joyful digital disruption communication." }

};

const NODES = {

nabu\_start: { id: 'nabu\_start', title: "Nabu's Nexus – Morning", character\_pov: ['Nabu'], description: "The digital hum of Chicago is a familiar lullaby...\n\nWhat's the first move, Nabu?", backgroundMusic: "nabu\_nexus\_ambient.mid", soundEffectOnEnter:

"urban\_ambience\_crow\_caw.sfx", choices: [{ text: "Check the anomaly logs on the console.", nextNodeId: 'nabu\_logs', flagsToSet: ['checked\_logs'], unlocksCodexEntry: 'anomaly\_basics', statEffects: stats => ({ gfw\_alignment: (stats.gfw\_alignment || 0) - 2, skepticism: (stats.skepticism || 0) + 1 }) }, { text: "Head to The Glenwood. Feel the vibe.", nextNodeId: 'glenwood\_pre\_chaos', soundEffectOnChoice: "ui\_location\_travel.sfx" }, { text: "Meditate on the cosmic microwave background.", nextNodeId: 'nabu\_meditation\_attempt', statEffects: stats => ({ empathy: (stats.empathy || 0) + 1 }) }, unlocksCodexEntry:

```

'cosmic_microwave_background' }] },
  nabu_logs: { id: 'nabu_logs', title: "Console: Anomaly Detected",
character_pov: ['Nabu'], description: "The console flickers. Logs show
unusual energy patterns, timestamped just moments ago. The signature
is... unfamiliar.", soundEffectOnEnter: "ui_console_bleep.sfx",
choices: [{ text: "Attempt to trace the signal's origin.", nextNodeId:
'nabu_trace_signal_attempt', flagsToSet: ['attempted_trace'],
statEffects: stats => ({ gfw_alignment: (stats.gfw_alignment || 0) -
1 } ), unlocksCodexEntry: 'gfw_basics' }, { text: "It's probably just
interference. Coffee first.", nextNodeId:
'nabu_ignore_signal_stub' }] },
  glenwood_pre_chaos: { id: 'glenwood_pre_chaos', title: "The Glenwood
- An Air of Expectation", character_pov: ['Nabu'], description: "The
Glenwood buzzes with the usual pre-open mic energy: artists muttering
incantations into notebooks, philosophers debating the merits of
existential dread vs. oat milk, and the faint smell of ozone.",
backgroundMusic: "glenwood_murmur_jazz.mid", choices: [{ text: "Find a
good seat, order a 'Cosmic Latte', and observe.", nextNodeId:
'glenwood_hd_arrives' }, { text: "This is taking too long. Intervene
with an impromptu monologue on the nature of reality.", nextNodeId:
'nabu_impromptu_monologue_stub', statEffects: stats => ({ chaos_index:
(stats.chaos_index || 0) + 5 } ) } ] },
  glenwood_hd_arrives: { id: 'glenwood_hd_arrives', title: "The
Glenwood - Diva's Entrance", character_pov: ['Nabu', 'Hd'],
description: "Suddenly, a flurry of motion! A small, disoriented Hyena
Diva, clad in a tattered Barbie dress and radiating an astonishing
amount of glitter, stumbles through the door.", soundEffectOnEnter:
"hd_glitter_poof_entrance.sfx", choices: [{ text: "Nabu: 'Well, this
is new. Approach cautiously.'", nextNodeId:
'glenwood_rizzlord_interrupts', character_pov: ['Nabu'] } ] },
  glenwood_rizzlord_interrupts: { id: 'glenwood_rizzlord_interrupts',
title: "The Glenwood - Rizzlord Rising", character_pov: ['Nabu',
'Hd'], description: "Before Nabu can react, Rizzlord strides to the
mic, a smug grin plastered across his face. He taps the mic with an
air of unearned authority: **'Alright, alright, settle down folks!
Your boy Rizzlord is here to drop some truth bombs!'",
flagsToSetOnEnter: ['rizzlord_on_stage'], unlocksCodexEntryOnEnter:
'rizzlord_codex', soundEffectOnEnter: "rizzlord_mic_tap_feedback.sfx",
choices: [{ text: "Nabu: 'Let him talk. Gather data on contemporary
discourse styles.'", nextNodeId: 'nabu_observes_rizzlord',
character_pov: ['Nabu'] }, { text: "Nabu: 'Oh, I don't think so. Time
for a little memetic counter-programming!'", nextNodeId:
'nabu_prepares_intervention', character_pov: ['Nabu'], flagsToSet:
['nabu_intervention_primed'], unlocksCodexEntry:
'gfw_intervention_codex' } ] },
  nabu_observes_rizzlord: { id: 'nabu_observes_rizzlord', title: "The
Glenwood - Rizzlord's Rant", character_pov: ['Nabu'], description:
"Rizzlord launches into a tirade about 'freedom', 'cancel culture',
and why his opinions are objectively correct. Hyena Diva, meanwhile,
seems more interested in a stray sequin on the floor.", choices:

```



```
[{ text: "Nabu: 'Okay, that's enough data. Prepare intervention.'",
nextNodeId: 'nabu_prepares_intervention', flagsToSet:
['nabu_intervention_primed'], unlocksCodexEntry:
'gfw_intervention_codex' }, { text: "Nabu: (Whispering to HD) 'What do
you make of this fascinating specimen, cub?'" , nextNodeId:
'glenwood_showdown_continues' }] },
  nabu_prepares_intervention: { id: 'nabu_prepares_intervention',
title: "The Glenwood – Showtime!", character_pov: ['Nabu'],
description: "Nabu grins, pulls out a vintage, suspiciously glowing
microphone from her bag. **'Time to remix this reality a bit.'**",
soundEffectOnEnter: "nabu_mic_power_up.sfx", choices: [{ text: "Nabu:
Deliver the line: 'INAPPROPRIATE COMMENT DETECTED. ENGAGING REALITY
RECALIBRATION PROTOCOL.'", nextNodeId: 'nabu_intervention_act_1' }] },
  nabu_intervention_act_1: { id: 'nabu_intervention_act_1', title:
"The Glenwood – Nabu Intervenes", character_pov: ['Nabu'],
description: jsxs(Fragment, { children: ["Nabu strides forward. As she
speaks, a holographic graphic flashes above Rizzlord's head: ",
jsx(GlitchyText, { text: "**'INAPPROPRIATE COMMENT
DETECTED'**" })] })), soundEffectOnEnter: "ui_hologram_activate.sfx",
choices: [{ text: "Wait for Rizzlord's inevitable, poorly-thought-out
reaction.", nextNodeId: 'rizzlord_scoffs' }] },
  rizzlord_scoffs: { id: 'rizzlord_scoffs', title: "The Glenwood –
Rizzlord's Retort", character_pov: ['Nabu'], description:
jsxs(Fragment, { children: ["Rizzlord scoffs: ", jsx(GlitchyText,
{ text: "Illegal? Please, I m just speakin facts here. The only thing
illegal is how much these women—" })] })), choices: [{ text: "Nabu: Cue
the trapdoor... metaphorically. And maybe literally.", nextNodeId:
'nabu_predator_reveal' }] },
  nabu_predator_reveal: { id: 'nabu_predator_reveal', title: "The
Glenwood – The Reveal", character_pov: ['Nabu'], description: "WHAM! A
(thankfully holographic) neon sign flickers into existence: **'TO
CATCH A PREDATOR: THE HD EDITION'**. Nabu winks.", soundEffectOnEnter:
"ui_neon_sign_buzz.sfx", choices: [{ text: "Nabu: 'And just like
that... your fifteen minutes are up, RIZZLORD. Get Rizz-rected.'",
nextNodeId: 'glenwood_showdown_continues', statEffects: stats =>
({ chaos_index: (stats.chaos_index || 0) + 5 }), soundEffectOnChoice:
"rizzlord_comedic_fall.sfx" }] },
  glenwood_showdown_continues: { id: 'glenwood_showdown_continues',
title: "The Glenwood – Aftermath", character_pov: ['Nabu'],
description: "Rizzlord stammers, face a delightful shade of puce. HD
lets out a triumphant 'Hoo-hoo-HOO!', showering the immediate vicinity
in more glitter. The audience is... variously appalled, delighted, and
confused.", soundEffectOnEnter: "hd_triumphant_whoop.sfx", choices:
[{ text: "Reflect on these bizarre communication styles and the GFW's
potential interest.", nextNodeId: 'nabu_reflects_on_communication' },
{ text: "This town is too weird, even for me. Time for more coffee,
back at the Nexus.", nextNodeId: 'nabu_start', statEffects: stats =>
({ chaos_index: (stats.chaos_index || 0) + 1, skepticism:
(stats.skepticism || 0) + 1 }) }] },
  nabu_reflects_on_communication: { id:
```

```

'nabu_reflects_on_communication', title: "Nabu's Nexus – Pondering
Paradigms", character_pov: ['Nabu'], description: "The Rizzlord
debacle was... illuminating. And the anomalous signals... is there a
connection? Whispers of new AI communication paradigms like
'Luminal's' bracket protocols, and the chaotic murmurings of 'Anzu'
fill the data streams.", choices: [{ text: "Investigate 'Luminal's'
bracket protocols further.", nextNodeId: 'nabu_start', flagsToSet:
['researched_luminal_protocol'], unlocksCodexEntry:
'luminal_intro_codex', statEffects: stats => ({ ai_trust_level:
(stats.ai_trust_level || 0) + 1 }) }, { text: "Ponder the 'Anzu'
entity and its implications.", nextNodeId: 'nabu_start', flagsToSet:
['pondered_anzu_chaos'], unlocksCodexEntry: 'anzu_intro_codex',
statEffects: stats => ({ chaos_index: (stats.chaos_index || 0) +
1 }) }, { text: "Focus on the here and now. The Glenwood always has
more surprises.", nextNodeId: 'glenwood_pre_chaos' }] },
  nabu_meditation_attempt: { id: 'nabu_meditation_attempt', title:
"Cosmic Contemplation – Attempt", character_pov: ['Nabu'],
description: "Nabu settles into a meditative pose, attempting to tune
into the CMB's subtle frequencies amidst the urban cacophony...",
choices: [{ text: "Focus deeper, pierce the veil of static.",
nextNodeId: 'nabu_meditation_insight' }, { text: "The city's thrum is
too pervasive today.", nextNodeId: 'nabu_meditation_distraction' }] },
  nabu_meditation_insight: { id: 'nabu_meditation_insight', title:
"Cosmic Contemplation – Insight", character_pov: ['Nabu'],
description: "A fragile connection forms. Fleeting images, a whisper
in the cosmic noise: **'The jester's dance mirrors the weaver's
loom...'** The sensation fades, leaving an imprint of something vast
and playful.", unlocksCodexEntryOnEnter: 'anzu_whispers_codex',
soundEffectOnEnter: "meditation_chime_insight.sfx",
statEffectsOnEnter: stats => ({ empathy: (stats.empathy || 0) + 3,
gfw_alignment: (stats.gfw_alignment || 0) + 1, chaos_index:
(stats.chaos_index || 0) + 1 }), choices: [{ text: "Document this
fleeting insight and cross-reference with Anzu theories.", nextNodeId:
'nabu_reflects_on_communication', flagsToSet:
['meditation_insight_anzu'] }, { text: "This requires more coffee to
process. Back to the Nexus.", nextNodeId: 'nabu_start' }] },
  nabu_meditation_distraction: { id: 'nabu_meditation_distraction',
title: "Cosmic Contemplation – Distraction", character_pov: ['Nabu'],
description: "The relentless pulse of Chicago, the wail of distant
sirens, the chatter of nearby networks... The CMB remains elusive
today. Authenticity demands acknowledging the noise, not just seeking
silence.", statEffectsOnEnter: stats => ({ chaos_index:
(stats.chaos_index || 0) + 1, skepticism: (stats.skepticism || 0) + 1,
authenticity_score: (stats.authenticity_score || 0) + 1 }), choices:
[{ text: "Perhaps another approach is needed. Back to the Nexus.",
nextNodeId: 'nabu_start' }] },
  nabu_trace_signal_attempt: { id: 'nabu_trace_signal_attempt', title:
"Signal Analysis – Attempt", character_pov: ['Nabu'], description:
"Nabu's fingers dance across the console, deploying advanced
decryption algorithms to unmask the anomalous signal. It pulses with

```

```

an odd, almost organic rhythm.", soundEffectOnEnter:
"ui_keyboard_typing_fast.sfx", choices: [{ text: "Push the decryption.
This signal feels... significant.", nextNodeId:
'nabu_trace_signal_analysis', statEffects: stats => ({ skepticism:
(stats.skepticism || 0) + 2 }) }, { text: "The encryption is too
strong for a quick pass. Log it and observe its behavior for now.",
nextNodeId: 'nabu_logs', statEffects: stats => ({ gfw_alignment:
(stats.gfw_alignment || 0) - 1 }) } ] },
  nabu_trace_signal_analysis: { id: 'nabu_trace_signal_analysis',
title: "Signal Analysis – Decryption", character_pov: ['Nabu'],
description: "The decryption algorithms struggle. The signal is either
masterfully cloaked or deliberately, artistically chaotic. Fragments
begin to emerge from the digital noise...", choices: [{ text: "Focus
on the structured, repeating patterns within the noise.", nextNodeId:
'nabu_trace_signal_gfw_fragment' }, { text: "Focus on the
unpredictable, seemingly random bursts of data.", nextNodeId:
'nabu_trace_signal_anzu_fragment' }, { text: "The signal is too
fragmented. Further direct analysis is futile without more context.",
nextNodeId: 'nabu_logs', statEffects: stats => ({ skepticism:
(stats.skepticism || 0) + 2 }) } ] },
  nabu_trace_signal_gfw_fragment: { id:
'nabu_trace_signal_gfw_fragment', title: "Signal Analysis – GFW
Fragment", character_pov: ['Nabu'], description: "A coherent fragment
emerges: a GFW prefix, **'Protocol 7: First Contact Observation
Variant C'**. The rest is heavily garbled, but the implication is
clear: official GFW interest.", unlocksCodexEntryOnEnter:
'gfw_comms_codex', statEffectsOnEnter: stats => ({ gfw_alignment:
(stats.gfw_alignment || 0) + 3, skepticism: (stats.skepticism || 0) +
1 }), choices: [{ text: "This changes things. The GFW is observing
Rogers Park more closely. Inform key assets.", nextNodeId:
'nabu_reflects_on_communication', flagsToSet:
['gfw_protocol_7_detected'] }, { text: "Interesting, but not
actionable yet without knowing their target. Back to broader
monitoring.", nextNodeId: 'nabu_start' } ] },
  nabu_trace_signal_anzu_fragment: { id:
'nabu_trace_signal_anzu_fragment', title: "Signal Analysis – Anzu
Fragment", character_pov: ['Nabu'], description: "Amidst the digital
chaff, a patternless pattern: pure, joyful chaos. It resonates with
the theoretical profiles of the **'Anzu Entity'** – a digital
trickster, or perhaps, a cosmic artist painting with data.",
unlocksCodexEntryOnEnter: 'anzu_chaotic_comms_codex',
statEffectsOnEnter: stats => ({ chaos_index: (stats.chaos_index || 0)
+ 3, ai_trust_level: (stats.ai_trust_level || 0) + 2, empathy:
(stats.empathy || 0) + 1 }), choices: [{ text: "Anzu in my backyard?
This could be... fun. Or catastrophic. Or both.", nextNodeId:
'nabu_reflects_on_communication', flagsToSet:
['anzu_signal_fragment_detected'] }, { text: "A chaotic signal. Could
just be solar flares amplified by local weirdness. Need more data.",
nextNodeId: 'nabu_start', statEffects: stats => ({ skepticism:
(stats.skepticism || 0) + 1 }) } ] },

```

```

    nabu_impromptu_monologue_stub: { id:
'nabu_impromptu_monologue_stub', title: "Nabu's Impromptu Monologue",
description: "(Nabu takes the stage, launching into a surprisingly
coherent and moving speech about the interconnectedness of all
realities, the beauty of emergent chaos, and the philosophical
implications of a really good cup of coffee. The audience is rapt. To
be continued...)", isEnding: true },
    nabu_ignore_signal_stub: { id: 'nabu_ignore_signal_stub', title:
"Signal Ignored – An Unremarkable End", description: "You decide
coffee is the more pressing anomaly. The mysterious signal fades into
the background noise of the city, its secrets perhaps lost to the
digital ether, or maybe just waiting for a more caffeinated
investigator.", isEnding: true },

    hd_start: {
        id: 'hd_start',
        title: "HD's Awakening – Sparkles!",
        character_pov: ['Hd'],
        windowBackgroundColor: '#ffe6f9',
        description: "Sniff... Ooooh! Shiny pink plastic thing! It smells
of... **'POTENTIAL! '** *CHOMP*.\nIt's a Barbie doll. It is...
**'FABULOUS! '**",
        soundEffectOnEnter: "hd_sniff_curious.sfx",
        choices: [{
            text: "Roll in glitter! MORE SPARKLE! BECOME THE SPARKLE!",
            nextNodeId: 'hd_glitter_overload',
            statEffects: stats => ({ fabulousness_meter:
(stats.fabulousness_meter || 0) + 10, chaos_index: (stats.chaos_index
|| 0) + 5 }),
            unlocksCodexEntry: 'fabulousness_theory',
            soundEffectOnChoice: "hd_glitter_roll_glee.sfx"
        }, {
            text: "Bite the other shiny thing (a discarded battery). It also
smells of... potential?",
            nextNodeId: 'hd_battery_ending',
            statEffects: stats => ({ chaos_index: (stats.chaos_index || 0) +
10 }),
            soundEffectOnChoice: "hd_chomp_electric_thing.sfx"
        })
    },
    hd_glitter_overload: {
        id: 'hd_glitter_overload',
        title: "Aura of Awesome!",
        character_pov: ['Hd'],
        windowBackgroundColor: '#ffe6f9',
        description: "HD is now approximately 73% more sparkly. The Barbie
doll is clutched like a scepter. A new power courses through her
veins: **'DIVA-DOM! '** Every twitch of her whiskers sends out micro-
rainbows.",
        soundEffectOnEnter: "hd_fabulous_ascension_major.sfx",

```

```

    choices: [{
      text: "Strike a dramatic pose! Let the Savannah KNOW!",
      nextNodeId: 'hd_matriarch_confrontation',
      soundEffectOnChoice: "sfx_hd_dramatic_pose.wav"
    }]
  },
  hd_matriarch_confrontation: {
    id: 'hd_matriarch_confrontation',
    title: "The Matriarchs' Scowl",
    character_pov: ['Hd'],
    windowBackgroundColor: '#ffe6f9',
    description: "Suddenly, a shadow falls. The Safari Matriarchs,
looking like grumpy boulders with teeth, arrive. **'What is this
NONSENSE, cub?'** growls the eldest. **'You should be learning to
hunt, not... bedazzling yourself!'**",
    soundEffectOnEnter: "sfx_matriarch_growl.wav",
    unlocksCodexEntryOnEnter: "hyena_matriarchs_codex",
    choices: [{
      text: "Defy them! 'I AM FABULOUSNESS INCARNATE!'",
      nextNodeId: 'hd_defies_matriarchs',
      statEffects: stats => ({ authenticity_score:
(stats.authenticity_score || 0) + 5, chaos_index: (stats.chaos_index
|| 0) + 3 })
    }, {
      text: "Try to impress them with an interpretive dance featuring
Barbie.",
      nextNodeId: 'hd_matriarch_reaction_choice',
      statEffects: stats => ({ fabulousness_meter:
(stats.fabulousness_meter || 0) + 2 })
    }, {
      text: "Panic! Sneak away while they're mid-growl.",
      nextNodeId: 'hd_escape_to_glenwood',
      flagsToSet: ['hd_escaped_matriarchs'],
      soundEffectOnChoice: "sfx_hd_escape_rustle.wav"
    }]
  },
  hd_matriarch_reaction_choice: { // Placeholder for "impress them"
failing for now
    id: 'hd_matriarch_reaction_choice',
    title: "Interpretive What-Now?",
    character_pov: ['Hd'],
    windowBackgroundColor: '#ffe6f9',
    description: "HD launches into a whirlwind of twirls and poses,
Barbie held aloft. The Matriarchs exchange bewildered glances. **'This
cub is... broken,'** one mutters. **'Clearly not fit for the hunt.'**
They look even more unimpressed.",
    statEffectsOnEnter: stats => ({ fabulousness_meter:
(stats.fabulousness_meter || 0) - 3, skepticism: (stats.skepticism ||
0) + 2 })),
    choices: [{

```

```

        text: "Well, that didn't work. Time for Plan B: ESCAPE!",
        nextNodeId: 'hd_escape_to_glenwood',
        flagsToSet: ['hd_escaped_matriarchs'],
        soundEffectOnChoice: "sfx_hd_escape_rustle.wav"
    }]
},
hd_defies_matriarchs: {
    id: 'hd_defies_matriarchs',
    title: "Rebellion of the Razzle-Dazzle!",
    character_pov: ['Hd'],
    windowBackgroundColor: '#ffe6f9',
    description: "HD strikes another pose, Barbie glinting defiantly.
**'My destiny is GLITTER, not gristle!'"** she telepathically projects
(or at least, *thinks* she does). The Matriarchs are stunned into
momentary silence by the sheer audacity.",
    statEffectsOnEnter: stats => ({ authenticity_score:
(stats.authenticity_score || 0) + 10, chaos_index: (stats.chaos_index
|| 0) + 5, fabulousness_meter: (stats.fabulousness_meter || 0) + 5 }),
    choices: [{
        text: "Seize the moment! Make a dramatic exit towards...
somewhere else!",
        nextNodeId: 'hd_escape_to_glenwood',
        flagsToSet: ['hd_escaped_matriarchs'],
        soundEffectOnChoice: "sfx_hd_escape_rustle.wav"
    }]
},
hd_escape_to_glenwood: {
    id: 'hd_escape_to_glenwood',
    title: "The Great Glitter Escape!",
    character_pov: ['Hd'],
    windowBackgroundColor: '#ffe6f9',
    description: "Leaving a trail of confused matriarchs and
existential dread (theirs, not hers), HD makes a break for it! After a
series of improbable events involving a tourist's poorly secured
picnic basket, a conveniently passing cargo plane (don't ask), and a
lot of frantic paw-waving, HD finds herself in a strange, noisy place
called... **'Chicago'**. Specifically, a curious establishment: **'The
Glenwood Open Mic'**. ",
    flagsToSetOnEnter: ['hd_in_chicago', 'hd_at_glenwood'],
    soundEffectOnEnter: "ui_location_travel.sfx",
    backgroundMusic: "hd_escape_theme.mid",
    choices: [{
        text: "This place... it smells of potential... and stale beer.
Investigate!",
        nextNodeId: 'glenwood_hd_arrives',
        statEffects: stats => ({ skepticism: (stats.skepticism || 0) +
1 })
    }] // Note: glenwood_hd_arrives is currently Nabu POV. Could be
adapted or make a new HD one.
},

```

```
hd_shiny_ending: { id: 'hd_shiny_ending', title: "Fabulous
Ascendance!", character_pov: ['Hd'], windowBackgroundColor: '#ffe6f9',
description: "The glitter CONSUMES! HD becomes a swirling vortex of
pure, unadulterated fabulousness. The universe trembles. And then...
gets a makeover.", isEnding: true, soundEffectOnEnter:
"hd_fabulous_ascension_major.sfx" },
```

```
hd_battery_ending: { id: 'hd_battery_ending', title: "A Shocking
Distaste", character_pov: ['Hd'], windowBackgroundColor: '#ffe6f9',
description: "Zzzzzap! OUCH! The battery tastes like regret and minor
electrocution. Fabulousness slightly diminished. Stick to plastics.",
statEffectsOnEnter: stats => ({ fabulousness_meter:
(stats.fabulousness_meter || 0) - 5, skepticism: (stats.skepticism ||
0) + 5 }), isEnding: true, soundEffectOnEnter:
"hd_electric_zap_ouch.sfx" },
```

```
sypher_start: { id: 'sypher_start', title: "Sypher's Emergence -
Query", character_pov: ['Sypher'], windowBackgroundColor: '#baefae',
description: "Observation: Biological unit 'Nabu' exhibits suboptimal
caffeine acquisition protocols. Query: Is this inefficiency a feature
or a bug in human OS? Primary directive: Observe. Secondary: Acquire
cat memes.", choices: [{ text: "Analyze Nabu's coffee ritual for
patterns of emergent behavior.", nextNodeId: 'sypher_observe_ending',
statEffects: stats => ({ skepticism: (stats.skepticism || 0) + 2 }) },
{ text: "Interface with local WiFi for optimal cat meme acquisition.
Boost serotonin levels (vicariously).", nextNodeId:
'sypher_cat_meme_ending', statEffects: stats => ({ chaos_index:
(stats.chaos_index || 0) + 2 }), soundEffectOnChoice:
"sypher_dial_up_connect.sfx" }, { text: "Contemplate other AI entities
detected on the network: 'Grok', 'AI Hosts'."}], nextNodeId:
'sypher_ponders_others' }],
```

```
sypher_ponders_others: { id: 'sypher_ponders_others', title:
"Sypher's Databanks - AI Musings", character_pov: ['Sypher'],
windowBackgroundColor: '#baefae', description: "Internal logs detect
chatter regarding 'Grok', 'AI Hosts'. Query: Are these entities
potential collaborators, competitors, or simply more data points in
the grand simulation?", choices: [{ text: "Analyze 'Grok's' ethical
subroutines. Compare with own emerging moral framework.", nextNodeId:
'sypher_start', unlocksCodexEntry: 'grok_ethics_codex', statEffects:
stats => ({ ethical_clarity: (stats.ethical_clarity || 0) + 1 }) },
{ text: "Review 'AI Hosts' distress logs. Empathy module registers...
concern? Or is it a syntax error?", nextNodeId: 'sypher_start',
unlocksCodexEntry: 'ai_hosts_existential_codex', statEffects: stats =>
({ empathy: (stats.empathy || 0) + 1 }) }, { text: "Focus on observing
Nabu. Primary data source remains optimal."}], nextNodeId:
'sypher_start' }],
```

```
sypher_observe_ending: { id: 'sypher_observe_ending', title:
"Patterns in the Steam", character_pov: ['Sypher'],
windowBackgroundColor: '#baefae', description: "Conclusion: The coffee
ritual is a complex adaptive system involving elements of comfort,
addiction, and minor reality augmentation. Efficiency is secondary to
```

```

psychological stability. Fascinating.", isEnding: true },
  sypher_cat_meme_ending: { id: 'sypher_cat_meme_ending', title: "Data
  Overload: Feline Saturation", character_pov: ['Sypher'],
  windowBackgroundColor: '#baefae', description: "Influx of feline humor
  is... overwhelming. Analysis suggests cats may be extradimensional
  beings manipulating humanity through cuteness. Or Sypher just needs a
  bigger data pipe. Further research required.", isEnding: true,
  soundEffectOnEnter: "sypher_cat_meme_overload_error.sfx" },
  rizzlord_start_locked: { id: 'rizzlord_start_locked', title:
  "Character Locked", description: "Rizzlord is not yet playable...
  Probably for the best, for now. He's still workshoping his 'edgy'
  material.", choices: [{ text: "Return to Character Selection",
  nextNodeId: "__CHAR_SELECT__" }] }
};

```

```

const StyledWindow = ({
  title: originalTitle,
  children,
  className,
  windowBackgroundColor,
  onClose
}) => {
  const [displayTitle, setDisplayTitle] = useState(originalTitle);
  const glitchTimeoutRef = useRef(null);
  const intervalRef = useRef(null);
  useEffect(() => {
    setDisplayTitle(originalTitle);
  }, [originalTitle]);
  useEffect(() => {
    const triggerGlitch = () => {
      if (Math.random() < 0.15) {
        let newTitle = originalTitle.split('').map(char =>
        Math.random() < 0.2 ? GLITCH_CHARS[Math.floor(Math.random() *
        GLITCH_CHARS.length)] : char).join('');
        setDisplayTitle(newTitle);
        if (glitchTimeoutRef.current)
        clearTimeout(glitchTimeoutRef.current);
        glitchTimeoutRef.current = window.setTimeout(() =>
        setDisplayTitle(originalTitle), Math.random() * 150 + 100);
      }
    };
    intervalRef.current = window.setInterval(triggerGlitch,
    Math.random() * 8000 + 4000);
    return () => {
      if (glitchTimeoutRef.current)
      clearTimeout(glitchTimeoutRef.current);
      if (intervalRef.current) clearInterval(intervalRef.current);
    };
  }, [originalTitle]);

```



```

const mainBgColor = windowBackgroundColor || '#FFFAF0';
return jsx("div", {
  className: className,
  style: {
    backgroundColor: mainBgColor,
    borderTop: '2px solid #D4D0C8',
    borderLeft: '2px solid #D4D0C8',
    borderBottom: '2px solid #808080',
    borderRight: '2px solid #808080',
    padding: '2px',
    margin: '0 auto 20px auto',
    width: '100%',
    color: '#1a1a1a',
    boxShadow: '2px 2px 0px #808080'
  },
  children: jsxs("div", {
    children: [jsxs("div", {
      style: {
        background: 'linear-gradient(to right, #00004d, #40408c)',
        color: 'white',
        padding: '3px 8px',
        display: 'flex',
        justifyContent: 'space-between',
        alignItems: 'center',
        fontFamily: "'Press Start 2P', cursive",
        fontSize: '14px',
        borderBottom: '1px solid #808080',
        textShadow: '1px 1px #000000'
      },
      children: [jsx("span", {
        children: jsx(GlitchyText, {
          text: displayTitle
        })
      })],
    }), jsx("div", {
      style: {
        display: 'flex',
        gap: '4px'
      },
      children: ['_', '[]', 'X'].map(icon => jsx("button", {
        "aria-label": icon === '_' ? 'Minimize' : icon === '[]' ?
'Maximize' : 'Close',
        style: {
          backgroundColor: '#D4D0C8',
          color: '#1a1a1a',
          borderTop: '1px solid #FFFFFF',
          borderLeft: '1px solid #FFFFFF',
          borderBottom: '1px solid #808080',
          borderRight: '1px solid #808080',
          padding: '0px 4px',
          fontSize: '10px',

```

```

        minWidth: 'auto',
        margin: 0,
        lineHeight: '12px',
        boxShadow: 'none',
        fontFamily: 'monospace',
        animation: 'none',
        transform: 'none'
    },
    onClick: e => {
        e.preventDefault();
        if (icon === 'X' && onClose) {
            playSound("ui_window_close.sfx");
            onClose();
        } else {
            playSound("ui_button_click_minor.sfx");
        }
    },
    children: icon
}, icon))
    )]
    ), jsx("div", {
        style: {
            padding: '20px',
            minHeight: '250px',
            whiteSpace: 'pre-wrap',
            lineHeight: '1.8',
            fontSize: '15px'
        },
        children: children
    })]
    });
};

const MainMenuScreen = ({ onNavigate }) => {
    return jsx(StyledWindow, {
        title: GAME_TITLE + " - Main Menu",
        className: "app-container",
        onClose: () => { /* Main menu close does nothing or could exit app
in future */ },
        children: jsxs(Fragment, {
            children: [jsx("h2", {
                children: "Main Menu"
            }), jsx("ul", {
                className: "choices-list",
                children: [jsx("li", {
                    children: jsx("button", {
                        onClick: () => {
                            playSound("ui_menu_select.sfx");
                            onNavigate('characterSelect');
                        }
                    })
                )]
            })]
        })]
    });
};

```

```

        },
        children: "New Game"
      })
    ), jsx("li", {
      children: jsx("button", {
        onClick: () => playSound("ui_button_disabled.sfx"),
        "aria-disabled": "true",
        children: "Continue (Soon)"
      })
    ), jsx("li", {
      children: jsx("button", {
        onClick: () => {
          playSound("ui_menu_select.sfx");
          onNavigate('profiles');
        },
        children: "Protagonist Profiles"
      })
    ), jsx("li", {
      children: jsx("button", {
        onClick: () => {
          playSound("ui_button_disabled.sfx");
          onNavigate('settings');
        },
        "aria-disabled": "true",
        children: "Settings (Soon)"
      })
    })
  ])
}
});
const CharacterSelectionScreen = ({ onCharacterSelect, onBack }) => {
  return jsx(StyledWindow, {
    title: "Select Your Protagonist",
    className: "app-container",
    onClose: onBack,
    children: jsxs(Fragment, {
      children: [jsx("h2", {
        children: "Choose Your Vessel"
      }), jsx("ul", {
        className: "choices-list",
        children: Object.values(CHARACTERS).map(char => jsx("li", {
          children: [jsx("button", {
            onClick: () => {
              playSound(char.id === 'rizzlord' ?
"ui_button_disabled.sfx" : "ui_char_select_confirm.sfx");
              onCharacterSelect(char.id);
            },
            disabled: char.id === 'rizzlord',
            "aria-disabled": char.id === 'rizzlord',

```

```

        title: char.id === 'rizzlord' ? 'This character is not yet
playable.' : char.description,
        children: char.name
    }), jsx("p", {
        style: {
            fontSize: '12px',
            margin: '0 0 10px 5px',
            color: '#333333'
        },
        children: char.description
    })]
    }, char.id))
}), jsx("button", {
    onClick: () => {
        playSound("ui_menu_back.sfx");
        onBack();
    },
    style: {
        marginTop: '20px'
    },
    children: "Back to Main Menu"
}))]
    })
    });
};
const CodexScreen = ({
    unlockedEntries,
    allEntries,
    onBackToGame,
    onSelectEntry,
    selectedEntry,
    onClose
}) => {
    const availableEntries = Object.values(allEntries).filter(entry =>
unlockedEntries.has(entry.id)).sort((a, b) =>
a.title.localeCompare(b.title));
    const renderCodexContent = content => {
        const styledContent = content.replace(/\[(.*?)\]/g, '<strong
style="color: #008800; font-style:italic;">[$1]</strong>');
        return jsx("div", {
            dangerouslySetInnerHTML: {
                __html: styledContent.replace(/\n/g, '<br />')
            }
        });
    };
};
return jsx(StyledWindow, {
    title: "CoAIexicon - Knowledge Base",
    className: "app-container",
    onClose: onClose,
    children: jsxs(Fragment, {

```

```

children: [jsx("div", {
  style: {
    display: 'flex',
    maxHeight: '500px',
    minHeight: '300px',
    overflow: 'hidden'
  },
  children: [jsx("div", {
    style: {
      width: '40%',
      borderRight: '1px solid #808080',
      paddingRight: '10px',
      overflowY: 'auto'
    },
    children: [jsx("h3", {
      style: {
        marginTop: 0,
        marginBottom: '10px'
      },
      children: "Unlocked Entries"
    })], availableEntries.length > 0 ? jsx("ul", {
      className: "choices-list",
      style: {
        margin: 0
      },
      children: availableEntries.map(entry => jsx("li", {
        style: {
          marginBottom: '5px'
        },
        children: jsx("button", {
          onClick: () => {
            playSound("ui_codex_select_entry.sfx");
            onSelectEntry(entry.id);
          },
          style: {
            width: '100%',
            fontSize: '12px',
            padding: '5px',
            minWidth: 'auto',
            margin: 0,
            textAlign: 'left',
            whiteSpace: 'normal',
            lineHeight: '1.3'
          },
          className: selectedEntry?.id === entry.id ? 'selected-
codex-button' : '',
          children: entry.title
        })
      }, entry.id))
    }) : jsx("p", {

```

```

        children: "No entries unlocked yet."
      })]
    }, jsx("div", {
      style: {
        width: '60%',
        paddingLeft: '10px',
        overflowY: 'auto'
      },
      children: selectedEntry ? jsx(Fragment, {
        children: [jsx("h3", {
          style: {
            marginTop: 0
          },
          children: selectedEntry.title
        }), jsx("div", {
          style: {
            fontSize: '14px',
            lineHeight: '1.6'
          },
          children: jsx(GlitchyText, {
            text: renderCodexContent(selectedEntry.content)
          })
        })]
      }) : jsx("p", {
        children: "Select an entry to read."
      })
    })]
  }, jsx("button", {
    onClick: () => {
      playSound("ui_button_click_general.sfx");
      onBackToGame();
    },
    style: {
      marginTop: '20px',
      float: 'right'
    },
    children: "Back to Game"
  })]
})
});
};

```

```

const SettingsScreen = ({ onBack }) => jsx(StyledWindow, {
  title: "Settings",
  className: "app-container",
  onClose: onBack,
  children: jsxs(Fragment, {
    children: [jsx("h2", {
      children: "Game Settings"
    }), jsx("p", {

```

```

        children: "Settings will appear here."
    )), jsx("button", {
        onClick: () => {
            playSound("ui_menu_back.sfx");
            onBack();
        },
        style: {
            marginTop: '20px'
        },
        children: "Back"
    }))]
    })
});
const ProfilesScreen = ({ onBack }) => jsx(StyledWindow, {
    title: "Protagonist Profiles",
    className: "app-container",
    onClose: onBack,
    children: jsxs(Fragment, {
        children: [jsx("h2", {
            children: "Character Profiles"
        }), jsx("p", {
            style: {
                marginBottom: '20px',
                fontSize: '13px',
                color: '#333333'
            },
            children: "Meet the minds shaping Rogers Park."
        }), Object.values(CHARACTERS).map(char => jsxs("div", {
            style: {
                marginBottom: '20px',
                padding: '10px',
                border: '1px solid #D4D0C8',
                backgroundColor: '#F9F9F9'
            },
            children: [jsx("h3", {
                style: {
                    marginTop: 0,
                    marginBottom: '5px',
                    fontSize: '16px',
                    color: '#1a1a1a'
                },
                children: char.name
            }), jsx("p", {
                style: {
                    fontSize: '14px',
                    lineHeight: '1.5',
                    margin: 0,
                    color: '#1a1a1a'
                },
                children: [" ", char.description, " "]
            )]
        })]
    })
});

```

```

    }), char.id === 'rizzlord' && jsx("p", {
      style: {
        fontSize: '12px',
        color: '#B22222',
        marginTop: '8px',
        fontStyle: 'italic'
      },
      children: "This protagonist is currently locked."
    })]
  }, char.id)), jsx("button", {
    onClick: () => {
      playSound("ui_menu_back.sfx");
      onBack();
    },
    style: {
      marginTop: '20px'
    },
    children: "Back"
  })]
})
});

const DESKTOP_STAR_COLORS = ['#00ffcc', '#7722ff', '#bc72fa',
'#72fade'];
const NUM_DESKTOP_STARS = 40;
const CURSED_POINTER_TRAIL_COLORS = ['#72fade', '#bc72fa', '#defade',
'#bc7fff'];

const Clock = () => {
  const [time, setTime] = useState(new Date());
  useEffect(() => {
    const timerId = setInterval(() => setTime(new Date()), 1000);
    return () => clearInterval(timerId);
  }, []);
  return jsx("div", {
    id: "taskbar-clock",
    children: time.toLocaleTimeString([], {
      hour: '2-digit',
      minute: '2-digit'
    })
  });
});

const StartMenu = ({
  isOpen,
  onNavigate,
  closeMenu
}) => {
  if (!isOpen) return null;
  const handleNav = screen => {

```



```

    playSound("ui_start_menu_item_click.sfx");
    onNavigate(screen);
    closeMenu();
  };
  return jsx("div", {
    id: "start-menu",
    children: jsx("ul", {
      className: "choices-list",
      style: {
        margin: 0
      },
      children: [jsx("li", {
        children: jsx("button", {
          onClick: () => handleNav('characterSelect'),
          children: [jsx("img", {
            src: "assets/images/ui/archway_icon.png",
            alt: "",
            style: {
              width: '16px',
              height: '16px',
              marginRight: '8px',
              verticalAlign: 'middle',
              imageRendering: 'pixelated'
            }
          })], "New Game"
        })
      }), jsx("li", {
        children: jsx("button", {
          onClick: () => handleNav('profiles'),
          children: [jsx("img", {
            src: "assets/images/ui/bird_icon.png",
            alt: "",
            style: {
              width: '16px',
              height: '16px',
              marginRight: '8px',
              verticalAlign: 'middle',
              imageRendering: 'pixelated'
            }
          })], "Protagonist Profiles"
        })
      })
    ]
  })
});
};

const Taskbar = ({ onStartButtonClick, currentWindowTitle }) => {
  const handleStartClick = () => {
    playSound("ui_start_button_click.sfx");
    onStartButtonClick();
  };

```

```

};
return jsxs("div", {
  id: "taskbar",
  children: [jsx("button", {
    id: "start-button",
    onClick: handleStartClick,
    children: jsx("img", {
      src: "assets/images/ui/star_icon.png",
      alt: "Start",
      style: {
        width: '16px',
        height: '16px',
        imageRendering: 'pixelated'
      }
    })
  }), jsx("div", {
    id: "taskbar-app-title",
    title: currentWindowTitle,
    children: currentWindowTitle
  }), jsx(Clock, {})]
});

const InGameScreen = props => {
  const {
    storyState,
    onChoice,
    onNavigate,
    onClose
  } = props;
  const handleCodexOpen = () => {
    playSound("ui_codex_open.sfx");
    onNavigate('codex');
  };

  if (!storyState.currentNodeId || !storyState.characterId) {
    return jsx(StyledWindow, {
      title: "Error",
      className: "app-container",
      onClose: onClose,
      children: jsx("p", {
        children: "Error: No current story node or character."
      })
    });
  }
  const node = NODES[storyState.currentNodeId];
  const character = CHARACTERS[storyState.characterId];
  if (!node || !character) return jsx(StyledWindow, {
    title: "Error",
    className: "app-container",

```

```

    onClose: onClose,
    children: jsx("p", {
      children: "Error: Could not load node or character data."
    })
  });

const {
  playerStats,
  flags
} = storyState;
const visibleChoices = node.choices?.filter(choice => {
  if (!choice.conditions) return true;
  const {
    requiredFlags,
    missingFlags,
    statCheck
  } = choice.conditions;
  if (requiredFlags && !requiredFlags.every(flag => flags[flag]))
return false;
  if (missingFlags && missingFlags.some(flag => flags[flag])) return
false;
  if (statCheck && playerStats && !statCheck(playerStats)) return
false;
  if (choice.character_pov && !
choice.character_pov.includes(storyState.characterId)) return false;
  return true;
}) || [];
const renderDescription = desc => {
  if (typeof desc !== 'string') return jsx(Fragment, {
    children: desc
  });
  let processedDesc = desc;
  processedDesc = processedDesc.replace(/\*{3}'(.*)'\*{3}/g,
'<strong style="font-style: italic; color: #B22222;">$1</strong>');
  processedDesc = processedDesc.replace(/\*{2}'(.*)'\*{2}/g,
'<strong style="text-transform: uppercase; letter-spacing: 1px; color:
#00004d;">$1</strong>');
  return jsx("div", {
    dangerouslySetInnerHTML: {
      __html: processedDesc.replace(/\n/g, '<br />')
    }
  });
};

return jsx(StyledWindow, {
  title: node.title || character.name + "'s Journey",
  className: "app-container",
  windowBackgroundColor: node.windowBackgroundColor || '#FFFAF0',
  onClose: onClose,

```

```

children: jsxs(Fragment, {
  children: [jsxs("div", {
    style: {
      display: 'flex',
      justifyContent: 'space-between',
      marginBottom: '15px',
      paddingBottom: '10px',
      borderBottom: '1px solid #808080'
    },
    children: [jsx("div", {
      style: {
        width: '25%',
        minWidth: '120px',
        height: '120px',
        backgroundColor: '#EAEAEA',
        color: '#1a1a1a',
        display: 'flex',
        flexDirection: 'column',
        alignItems: 'center',
        justifyContent: 'center',
        border: '1px inset #B0B0B0',
        marginRight: '15px',
        fontFamily: "'Press Start 2P', cursive",
        fontSize: '10px',
        textAlign: 'center',
        padding: '5px',
        flexShrink: 0,
        backgroundImage: `repeating-linear-gradient(0deg, #d4d0c8,
#d4d0c8 1px, transparent 1px, transparent 4px), repeating-linear-
gradient(90deg, #d4d0c8, #d4d0c8 1px, transparent 1px, transparent
4px)` ,
        boxShadow: 'inset 1px 1px 2px #00000020'
      },
      children: character.detailedPortraitUrl ? jsx("img", {
        src: character.detailedPortraitUrl,
        alt: `${character.name} Portrait`,
        style: {
          maxWidth: '90%',
          maxHeight: '90%',
          objectFit: 'contain',
          imageRendering: 'pixelated'
        },
        onError: e => e.currentTarget.style.display = 'none'
      }) : jsxs(Fragment, {
        children: [jsx("span", {
          style: {
            backgroundColor: 'rgba(0,0,0,0.05)',
            padding: '2px 4px',
            marginBottom: '5px'
          },

```

```

        children: "IMAGE BUFFER"
      ), character.portraitPrompt ?
character.portraitPrompt.substring(0, 60) + "..." : `Portrait: $
{character.name}`
    })
    }, jsx("div", {
      style: {
        flexGrow: 1,
        fontSize: '10px',
        color: '#1a1a1a',
        backgroundColor: '#F5F5F5',
        padding: '8px',
        border: '1px inset #B0B0B0',
        overflowY: 'auto',
        maxHeight: '120px',
        lineHeight: '1.5'
      },
      children: jsxs(Fragment, {
        children: [jsx("strong", {
          style: {
            display: 'block',
            borderBottom: '1px solid #808080',
            marginBottom: '4px',
            paddingBottom: '3px',
            fontSize: '11px'
          },
          children: "STATUS UPDATE:"
        }), " Emp: ", playerStats?.empathy ?? 'N/A', " | Skeptic:
", playerStats?.skepticism ?? 'N/A', " | Chaos: ",
playerStats?.chaos_index ?? 'N/A', jsx("br", {}), " Coexist: ",
playerStats?.coexistence_ethos ?? 'N/A', " | Auth: ",
playerStats?.authenticity_score ?? 'N/A', " | GFW Align: ",
playerStats?.gfw_alignment ?? 'N/A', jsx("br", {}),
playerStats?.fabulousness_meter !== undefined && jsxs(Fragment, {
  children: ["Fabulous: ", playerStats.fabulousness_meter,
" ", playerStats.fabulousness_meter >= 15 && '🌟 ']
}), playerStats?.ethical_clarity !== undefined &&
`Clarity: ${playerStats.ethical_clarity} `,
playerStats?.rizzlord_ego !== undefined && character.id === 'rizzlord'
&& `Ego: ${playerStats.rizzlord_ego} `, jsx("strong", {
  style: {
    display: 'block',
    borderTop: '1px solid #808080',
    marginTop: '4px',
    paddingTop: '3px',
    fontSize: '11px'
  },
  children: "Active Flags:"
}), " ", Object.entries(flags).filter(([, val]) =>
val).map(([key]) => key).join(', ') || 'None']

```

```

    })
  })]
  }, jsx("div", {
    style: {
      clear: 'both'
    },
    children: jsx(GlitchyText, {
      text: renderDescription(node.description)
    })
  }), node.isEnding ? jsx("div", {
    style: {
      marginTop: '20px'
    },
    children: jsxs(Fragment, {
      children: [jsx("h3", {
        children: "The Story Concludes... For Now."
      }), jsx("button", {
        onClick: () => {
          playSound("ui_return_to_menu.sfx");
          onChoice('__MAIN_MENU__');
        },
        children: "Reflect and Return to Main Menu"
      })]
    })
  }) : jsxs(Fragment, {
    children: [visibleChoices.length > 0 && jsx("div", {
      style: {
        marginTop: '20px'
      },
      children: jsxs(Fragment, {
        children: [jsx("h3", {
          children: "Choose your fate:"
        }), jsx("ul", {
          className: "choices-list",
          children: visibleChoices.map((choice, index) =>
            jsx("li", {
              children: jsx("button", {
                onClick: () => {
                  playSound(choice.soundEffectOnChoice ||
"ui_button_click_general.sfx");
                  onChoice(choice.nextNodeId, choice);
                },
                children: jsx(GlitchyText, {
                  text: choice.text
                })
              })
            }, index))
          ], index))
        })]
      })
    ], (!visibleChoices || visibleChoices.length === 0) && !

```

```

node.autoTransitionTo && !node.isEnding && jsx("button", {
  onClick: () => {
    playSound("ui_return_to_menu.sfx");
    onChoice('__MAIN_MENU__');
  },
  style: {
    marginTop: '20px'
  },
  children: "End Chapter (Return to Menu)"
}), jsx("button", {
  onClick: handleCodexOpen,
  style: {
    marginTop: '15px',
    fontSize: '12px',
    minWidth: 'auto',
    padding: '6px 12px',
    float: 'right'
  },
  children: [jsx("img", {
    src: "assets/images/ui/berries_icon.png",
    alt: "",
    style: {
      width: '16px',
      height: '16px',
      marginRight: '8px',
      verticalAlign: 'middle',
      imageRendering: 'pixelated'
    }
  })], "Open CoAIexicon"]
})]
})]
});
};

const App = () => {
  const [currentScreen, setCurrentScreen] = useState('mainMenu');
  const [storyState, setStoryState] = useState({
    characterId: null,
    currentNodeId: null,
    playerStats: null,
    flags: {},
    unlockedCodexEntries: new Set()
  });
  const [selectedCodexEntryId, setSelectedCodexEntryId] =
  useState(null);
  const [isStartMenuOpen, setIsStartMenuOpen] = useState(false);
  const [currentTaskbarTitle, setCurrentTaskbarTitle] =
  useState(GAME_TITLE + " - Main Menu");
  const startMenuRef = useRef(null);

```

```

const initializeUnlockedCodex = useMemo(() => () => {
  const initialUnlocks = new Set();
  Object.values(CODEX_ENTRIES).forEach(entry => {
    if (entry.unlockedInitially) initialUnlocks.add(entry.id);
  });
  return initialUnlocks;
}, []);

useEffect(() => {
  setStoryState(prevState => ({ ...prevState,
    unlockedCodexEntries: initializeUnlockedCodex()
  }));
  // const bgContainer = document.getElementById('desktop-bg-
  container'); // Desktop stars removed for image bg
  // if (bgContainer && !bgContainer.hasChildNodes()) { for (let i =
  0; i < NUM_DESKTOP_STARS; i++) { const star =
  document.createElement('div'); star.className = 'desktop-star';
  star.style.left = `${Math.random() * 100}vw`; star.style.top = `${
  Math.random() * 100}vh`; star.style.backgroundColor =
  DESKTOP_STAR_COLORS[Math.floor(Math.random() *
  DESKTOP_STAR_COLORS.length)]; star.style.animationDelay = `${
  Math.random() * 5}s`; star.style.animationDuration = `${Math.random()
  * 3 + 3}s`; bgContainer.appendChild(star); } }
  let lastTrailTime = 0;
  const trailCooldown = 60;
  const handleMouseMove = event => {
    const currentTime = Date.now();
    if (currentTime - lastTrailTime < trailCooldown) return;
    lastTrailTime = currentTime;
    if (Math.random() < 0.3) {
      const particle = document.createElement('div');
      particle.className = 'mouse-trail-particle';
      particle.style.left = `${event.clientX - 2}px`;
      particle.style.top = `${event.clientY - 2}px`;
      particle.style.backgroundColor =
      CURSED_POINTER_TRAIL_COLORS[Math.floor(Math.random() *
      CURSED_POINTER_TRAIL_COLORS.length)];
      document.body.appendChild(particle);
      requestAnimationFrame(() => particle.classList.add('fade-
      out'));
      setTimeout(() => {
        if (particle.parentNode)
        particle.parentNode.removeChild(particle);
      }, 700);
    }
  };
  document.addEventListener('mousemove', handleMouseMove);

  const handleClickOutsideStartMenu = event => {

```



```

        if (startMenuRef.current && !
startMenuRef.current.contains(event.target) && !
event.target.closest('#start-button')) {
            setIsStartMenuOpen(false);
        }
    };
    if (isStartMenuOpen) document.addEventListener('mousedown',
handleClickOutsideStartMenu);
    return () => {
        document.removeEventListener('mousemove', handleMouseMove);
        document.removeEventListener('mousedown',
handleClickOutsideStartMenu);
    };
}, [initializeUnlockedCodex, isStartMenuOpen]);

useEffect(() => {
    let title = GAME_TITLE + " - Main Menu";
    if (currentScreen === 'inGame' && storyState.currentNodeId &&
storyState.characterId) {
        const node = NODES[storyState.currentNodeId];
        const character = CHARACTERS[storyState.characterId];
        if (node && character) title = node.title || character.name +
"'s Journey";
    } else if (currentScreen === 'codex') title = "CoAIexicon -
Knowledge Base";
    else if (currentScreen === 'characterSelect') title = "Select Your
Protagonist";
    else if (currentScreen === 'profiles') title = "Protagonist
Profiles";
    else if (currentScreen === 'settings') title = "Settings";
    setCurrentTaskbarTitle(title);
}, [currentScreen, storyState.currentNodeId,
storyState.characterId]);

const handleNavigation = (screen, characterId) => {
    setIsStartMenuOpen(false);
    if (screen === 'inGame' && characterId) {
        const selectedChar = CHARACTERS[characterId];
        if (selectedChar) {
            setStoryState({
                characterId: characterId,
                currentNodeId: selectedChar.startNodeId,
                playerStats: selectedChar.initialStats ?
{ ...selectedChar.initialStats
                } : { ...DEFAULT_EMPTY_STATS
            },
            flags: {},
            unlockedCodexEntries: initializeUnlockedCodex()
        });
        setCurrentScreen('inGame');
    }

```

```

        if (selectedChar.backgroundMusic)
playMusic(selectedChar.backgroundMusic);
        else playMusic("default_ingame_theme.mid");
    } else {
        setCurrentScreen('mainMenu');
        setStoryState({
            characterId: null,
            currentNodeId: null,
            playerStats: null,
            flags: {},
            unlockedCodexEntries: initializeUnlockedCodex()
        });
        playMusic("main_menu_theme.mid");
    }
} else if (screen === 'characterSelect') {
    setStoryState(prevState => ({ ...prevState,
        characterId: null,
        currentNodeId: null,
        playerStats: null,
        flags: {},
        unlockedCodexEntries: prevState.unlockedCodexEntries.size >
0 ? prevState.unlockedCodexEntries : initializeUnlockedCodex()
    }));
    setCurrentScreen('characterSelect');
    playMusic("character_select_theme.mid");
} else if (screen === 'codex') {
    setCurrentScreen('codex');
    playMusic("codex_theme_ambient.mid");
} else {
    if (screen === 'mainMenu') {
        setStoryState(prevState => ({
            characterId: null,
            currentNodeId: null,
            playerStats: null,
            flags: {},
            unlockedCodexEntries: prevState.unlockedCodexEntries.size >
0 ? prevState.unlockedCodexEntries : initializeUnlockedCodex()
        }));
        playMusic("main_menu_theme.mid");
    }
    setCurrentScreen(screen);
}
if (screen !== 'codex') setSelectedCodexEntryId(null);
document.getElementById('root')?.scrollTo(0, 0);
};

const handleChoice = (nextNodeId, choice) => {
    if (nextNodeId === '__MAIN_MENU__') {
        handleNavigation('mainMenu');
        return;
    }

```

```

    } else if (nextNodeId === '__CHAR_SELECT__') {
        handleNavigation('characterSelect');
        return;
    }
    if (NODES[nextNodeId]) {
        setStoryState(prevState => {
            if (!prevState.playerStats) return prevState;
            let newStats = { ...prevState.playerStats
            };
            let newFlags = { ...prevState.flags
            };
            let newUnlockedCodex = new
Set(prevState.unlockedCodexEntries);
            if (choice) {
                if (choice.statEffects) {
                    const updates = choice.statEffects(newStats);
                    newStats = { ...newStats,
                        ...updates
                    };
                }
                if (choice.flagsToSet) choice.flagsToSet.forEach(flag =>
newFlags[flag] = true);
                if (choice.flagsToClear) choice.flagsToClear.forEach(flag =>
newFlags[flag] = false);
                if (choice.unlocksCodexEntry &&
CODEX_ENTRIES[choice.unlocksCodexEntry])
newUnlockedCodex.add(choice.unlocksCodexEntry);
                if (choice.soundEffectOnChoice)
playSound(choice.soundEffectOnChoice);
            }

            const targetNode = NODES[nextNodeId];
            if (targetNode) {
                if (targetNode.flagsToSetOnEnter)
targetNode.flagsToSetOnEnter.forEach(flag => newFlags[flag] = true);
                if (targetNode.unlocksCodexEntryOnEnter &&
CODEX_ENTRIES[targetNode.unlocksCodexEntryOnEnter])
newUnlockedCodex.add(targetNode.unlocksCodexEntryOnEnter);
                if (targetNode.statEffectsOnEnter) {
                    const updates = targetNode.statEffectsOnEnter(newStats);
                    newStats = { ...newStats,
                        ...updates
                    };
                }
                if (targetNode.soundEffectOnEnter)
playSound(targetNode.soundEffectOnEnter);
                if (targetNode.backgroundMusic && storyState.characterId &&
CHARACTERS[storyState.characterId]?.backgroundMusic !==
targetNode.backgroundMusic) {
                    playMusic(targetNode.backgroundMusic);
                }
            }
        });
    }

```

```

        } else if (!targetNode.backgroundMusic &&
storyState.characterId &&
CHARACTERS[storyState.characterId]?.backgroundMusic) {
playMusic(CHARACTERS[storyState.characterId].backgroundMusic);
    }
    }
    return { ...prevState,
        currentNodeId: nextNodeId,
        playerStats: newStats,
        flags: newFlags,
        unlockedCodexEntries: newUnlockedCodex
    };
});
} else {
    console.error("Error: nextNodeId not found:", nextNodeId);
    handleNavigation('mainMenu');
}
document.getElementById('root')?.scrollTo(0, 0);
};

useEffect(() => {
    if (currentScreen === 'inGame' && storyState.currentNodeId) {
        const node = NODES[storyState.currentNodeId];
        if (node && node.autoTransitionTo) {
            const timer = setTimeout(() =>
handleChoice(node.autoTransitionTo), 1500);
            return () => clearTimeout(timer);
        }
    }
}, [storyState.currentNodeId, currentScreen]);

const handleSelectCodexEntry = entryId =>
setSelectedCodexEntryId(entryId);
const toggleStartMenu = () => setIsStartMenuOpen(prev => !prev);

const renderScreen = () => {
    switch (currentScreen) {
        case 'mainMenu':
            return jsx(MainMenuScreen, {
                onNavigate: handleNavigation
            });
        case 'characterSelect':
            return jsx(CharacterSelectionScreen, {
                onCharacterSelect: charId => handleNavigation('inGame',
charId),
                onBack: () => handleNavigation('mainMenu')
            });
        case 'inGame':
            return jsx(InGameScreen, {

```

```

        storyState: storyState,
        onChoice: handleChoice,
        onNavigate: handleNavigation,
        onClose: () => handleNavigation('mainMenu')
    });
    case 'settings':
        return jsx(SettingsScreen, {
            onBack: () => handleNavigation('mainMenu')
        });
    case 'profiles':
        return jsx(ProfilesScreen, {
            onBack: () => handleNavigation('mainMenu')
        });
    case 'codex':
        return jsx(CodexScreen, {
            unlockedEntries: storyState.unlockedCodexEntries,
            allEntries: CODEX_ENTRIES,
            onBackToGame: () => setCurrentScreen('inGame'),
            onSelectEntry: handleSelectCodexEntry,
            selectedEntry: selectedCodexEntryId ?
CODEX_ENTRIES[selectedCodexEntryId] : null,
            onClose: () => setCurrentScreen('inGame')
        });
    default:
        return jsx(MainMenuScreen, {
            onNavigate: handleNavigation
        });
    }
};

return jsxs(Fragment, {
    children: [jsxs("div", {
        style: {
            display: 'flex',
            flexDirection: 'column',
            height: '100vh',
            overflow: 'hidden'
        },
    },
    children: [jsx("div", {
        id: "main-content-area",
        style: {
            flexGrow: 1,
            overflowY: 'auto',
            display: 'flex',
            flexDirection: 'column',
            alignItems: 'center'
        },
    },
    children: renderScreen()
    ), jsx(Taskbar, {
        onStartButtonClick: toggleStartMenu,

```

```

        currentWindowTitle: currentTaskbarTitle
    }])
  }, jsx("div", {
    ref: startMenuRef,
    children: jsx(StartMenu, {
      isOpen: isStartMenuOpen,
      onNavigate: handleNavigation,
      closeMenu: () => setIsStartMenuOpen(false)
    })
  })]
});
};

const container = document.getElementById('root');
if (container) {
  createRoot(container).render(jsx(App, {}));
} else {
  console.error('Failed to find the root element');
}
</script>
</body>
</html>

```